



SISTEMA DE CADASTRO EM JAVA COM PERSISTÊNCIA EM ARQUIVOS

Carlos Daniel Ferreira Diamantino 202501276131

POLO CENTRO - PORANGATU – GO
Programação Back-end com Java 2026.3

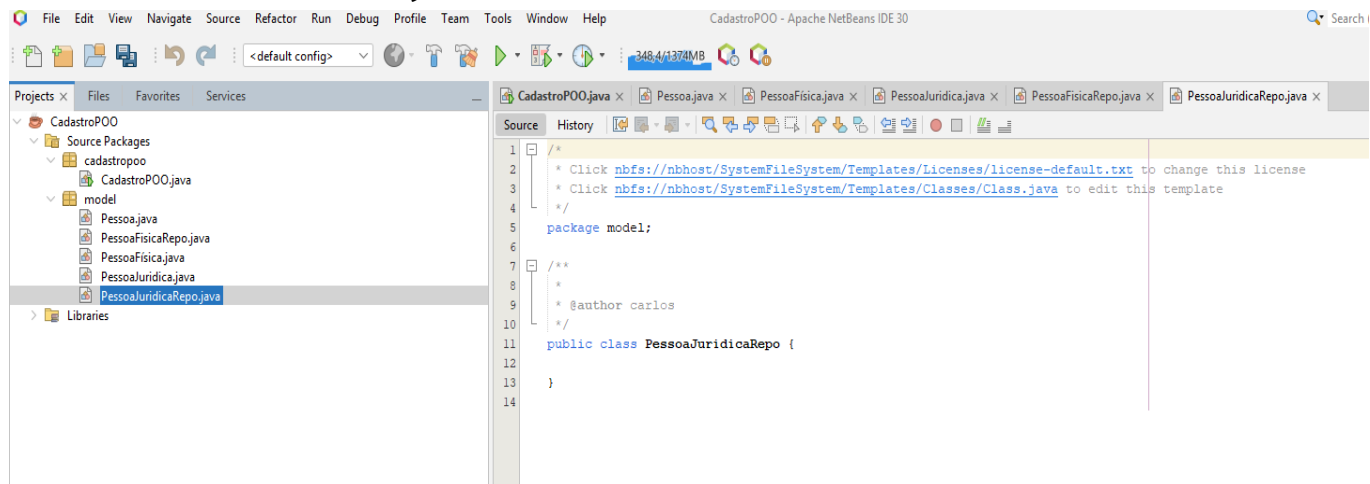
Repositório GitHub: <https://github.com/carlosdaniel045/Programacao-back-end-com-java>

Objetivo da Prática

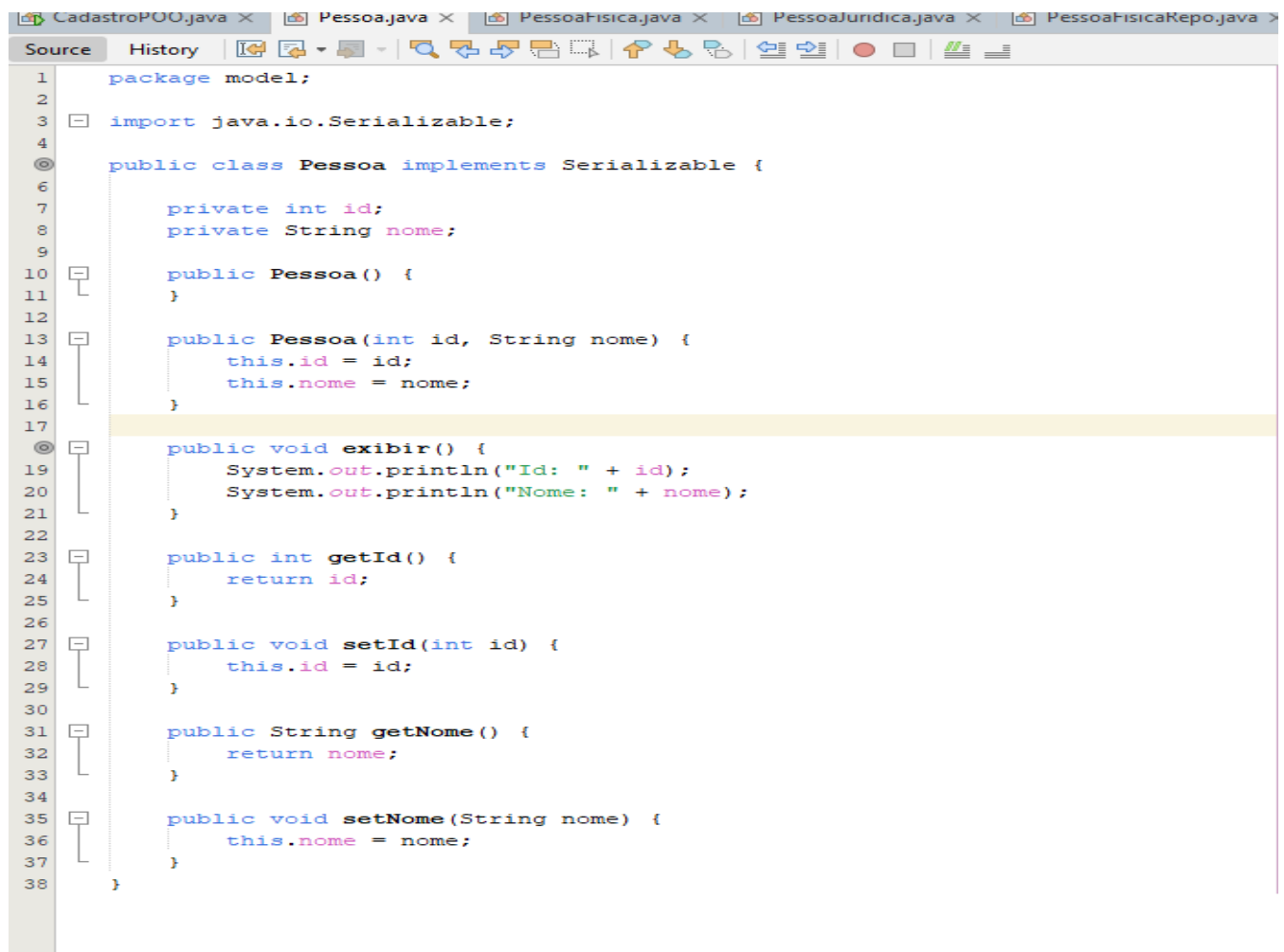
O objetivo desta prática foi aplicar os conceitos fundamentais da Programação Orientada a Objetos utilizando Java, implementando herança, polimorfismo, serialização e persistência de dados em arquivos binários.

1º Procedimento | Criação das Entidades e Sistema de Persistência

1º ESTRUTURA DO PROJETO



2º CLASSES E REPOSITÓRIOS



```
CadastroPOO.java x Pessoa.java x PessoaFisica.java x PessoaJuridica.java x PessoaFisicaRepo.java x
Source History
1 package model;
2
3 import java.io.Serializable;
4
5 public class PessoaJuridica extends Pessoa implements Serializable {
6
7     private String cnpj;
8
9     public PessoaJuridica() {
10     }
11
12     public PessoaJuridica(int id, String nome, String cnpj) {
13         super(id, nome);
14         this.cnpj = cnpj;
15     }
16
17     @Override
18     public void exibir() {
19         super.exibir();
20         System.out.println("CNPJ: " + cnpj);
21     }
22
23     public String getCnpj() {
24         return cnpj;
25     }
26
27     public void setCnpj(String cnpj) {
28         this.cnpj = cnpj;
29     }
30 }
```

```
CadastroPOO.java x Pessoa.java x PessoaFisica.java x PessoaJuridica.java x PessoaFisicaRepo.java x
Source History
1 package model;
2
3 import java.io.Serializable;
4
5 public class PessoaFisica extends Pessoa implements Serializable {
6
7     private String cpf;
8     private int idade;
9
10    public PessoaFisica() {
11    }
12
13    public PessoaFisica(int id, String nome, String cpf, int idade) {
14        super(id, nome);
15        this.cpf = cpf;
16        this.idade = idade;
17    }
18
19    @Override
20    public void exibir() {
21        super.exibir();
22        System.out.println("CPF: " + cpf);
23        System.out.println("Idade: " + idade);
24    }
25
26    public String getCpf() {
27        return cpf;
28    }
29
30    public void setCpf(String cpf) {
31        this.cpf = cpf;
32    }
33
34    public int getIdade() {
35        return idade;
36    }
37
38    public void setIdade(int idade) {
39        this.idade = idade;
40    }
41 }
```

3º EXECUÇÃO DO PROCEDIMENTO 1

The screenshot displays an IDE interface with the following components:

- Source Packages:** A tree view showing the project structure. The 'CadastroPOO' package contains a 'model' sub-package with files: 'Pessoa.java', 'PessoaFisica.java', 'PessoaFisicaRepo.java', 'PessoaJuridica.java', and 'PessoaJuridicaRepo.java'. There are also 'Test Packages', 'Libraries', and 'Test Libraries'.
- main - Navigator:** A view showing the 'main' method in the 'main' class, with the signature 'main(String[] args)'.
- Source:** The source code of 'CadastroPOO.java' is displayed. It includes a 'main' method that creates and manages repositories for physical and legal persons, inserts data, and prints the results.
- Output - CadastroPOO (run):** The output of the program is shown, displaying the details of two physical persons and the header for legal persons.

```
public class CadastroPOO {  
    public static void main(String[] args) {  
        try {PessoaFisicaRepo repol = new PessoaFisicaRepo();  
            repol.inserir(new PessoaFisica(1, "Carlos", "11111111111", 20));  
            repol.inserir(new PessoaFisica(2, "Maria", "22222222222", 25));  
            repol.persistir("pessoasfisicas.bin");  
            PessoaFisicaRepo repo2 = new PessoaFisicaRepo();  
            repo2.recuperar("pessoasfisicas.bin");  
            System.out.println("PESSOAS FISICAS");  
            for (PessoaFisica pf : repo2.obterTodos()) {  
                pf.exibir();  
                System.out.println();  
            }  
            PessoaJuridicaRepo repo3 = new PessoaJuridicaRepo();  
            repo3.inserir(new PessoaJuridica(1, "Empresa A", "11111111000111"));  
            repo3.inserir(new PessoaJuridica(2, "Empresa B", "22222222000122"));  
            repo3.persistir("pessoasjuridicas.bin");  
            PessoaJuridicaRepo repo4 = new PessoaJuridicaRepo();  
            repo4.recuperar("pessoasjuridicas.bin");  
            System.out.println("PESSOAS JURIDICAS");  
        }  
    }  
}
```

run:
PESSOAS FISICAS
Id: 1
Nome: Carlos
CPF: 11111111111
Idade: 20

Id: 2
Nome: Maria
CPF: 22222222222
Idade: 25
PESSOAS JURIDICAS

Análise e Conclusão do 1º Procedimento

a) Vantagens e desvantagens da herança.

A principal vantagem da herança é o reaproveitamento de código, permitindo que classes derivadas utilizem atributos e métodos já definidos em uma classe base. Isso reduz duplicações e facilita a manutenção do sistema. Como desvantagem, existe um maior acoplamento entre as classes, pois alterações realizadas na classe pai podem impactar diretamente as classes filhas.

b) Por que Serializable é necessária?

A interface Serializable permite que objetos sejam convertidos em uma sequência de bytes. Dessa forma, os objetos podem ser armazenados em arquivos binários e posteriormente recuperados pelo sistema. Sem a implementação dessa interface, o Java não permite a serialização dos objetos.

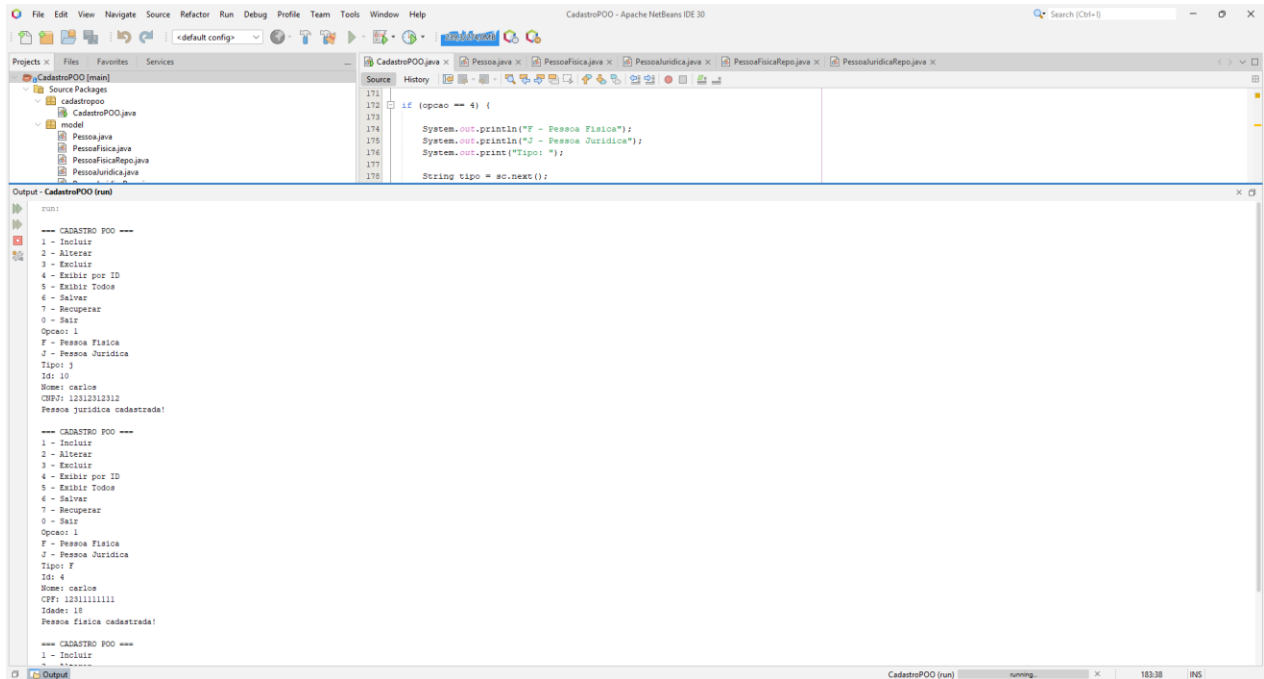
c) Como a API Stream utiliza o paradigma funcional?

A API Stream utiliza conceitos do paradigma funcional através de expressões lambda e operações como `filter()`, `map()` e `forEach()`. Esse modelo permite manipular coleções de forma mais simples e declarativa, tornando o código mais legível e reduzindo a necessidade de estruturas repetitivas.

d) Qual padrão é adotado na persistência?

Durante a persistência dos dados foi adotada uma abordagem semelhante ao padrão Repository, onde as entidades representam os dados e os repositórios ficam responsáveis pelas operações de armazenamento e recuperação. Essa separação melhora a organização e facilita a manutenção do sistema.

2º Procedimento | Criação do Cadastro em Modo Texto



Análise e Conclusão do 2º Procedimento

a) O que são elementos estáticos?

Elementos estáticos pertencem à classe e não a um objeto específico. O método main é declarado como estático porque ele representa o ponto de entrada da aplicação e precisa ser executado sem a necessidade de instanciar a classe principal.

b) Para que serve a classe Scanner?

A classe Scanner é utilizada para leitura de dados fornecidos pelo usuário através do teclado. Ela permite capturar diferentes tipos de informações, como textos, números inteiros e números decimais, sendo muito utilizada em aplicações executadas pelo console.

c) Como os repositórios ajudaram na organização?

As classes de repositório permitiram separar as responsabilidades do sistema. As entidades ficaram responsáveis apenas pelos dados, enquanto os repositórios ficaram responsáveis pelas operações de manipulação e persistência. Essa separação tornou o projeto mais organizado, reutilizável e fácil de manter.

Conclusão

O desenvolvimento desta prática permitiu aplicar diversos conceitos fundamentais da programação orientada a objetos utilizando Java. Foram implementados recursos como herança, polimorfismo, serialização de objetos, persistência em arquivos binários e interação com o usuário através de menus em modo texto. A utilização de repositórios contribuiu para uma melhor organização do código, enquanto o uso da persistência possibilitou armazenar informações de forma permanente. Dessa forma, a atividade contribuiu significativamente para a consolidação dos conhecimentos relacionados ao desenvolvimento de aplicações Java utilizando boas práticas de programação.